

Simon Grundner
k12136610

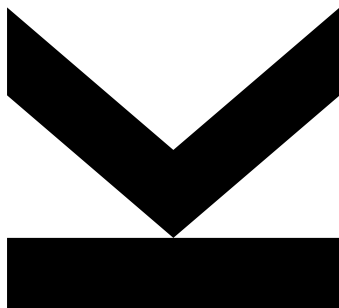
Institute for Integrated
Circuits and Quantum
Computing

@ k12136610@stu-
dents.jku.at

https://github.com/s-
grundner/LVA-EIC-KV

January 17, 2026

Tiny Tapeout - MIDI Polyphonic Synthesizer



Technical Report

KV Integrated Circuits Design - WiSe25



JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Abstract MIDI Polyphonic Synthesizer on the Tiny Tapeout ASIC

Contents

Abstract	ii
Acronyms	iii
1. Introduction	1
2. Digital Instruments and Music Theory	2
2.1 The MIDI-Protocol	2
2.2 Instrument Tuning	3
2.3 Instrument Characteristics	3
2.4 Polyphony	4
2.5 MIDI Physical Layer	5
3. System Overview	7
3.1 Pinout and Top Level Module	7
3.2 Block Diagram	7
3.3 Layout	8
3.4 Global Settings	9
3.5 Modules	10
4. Receiver	11
4.1 Statemachine	11
5. MIDI-Decoder	14
5.1 Statemachine	14
5.2 Limitations	16
6. Synthesizer	17
6.1 Frequency Calculation and Generation	17
6.2 Oscillatorstack	17
7. System Testing	18
8. External Hardware	18
8.1 MIDI-Source	18
8.2 Input Side	18
8.3 Output Side	18
9. Used Software	18
References	19

Abstract

The project on hand implements a polyphonic audio square wave synthesizer in Verilog, which can be controlled via the musical instrument digital interface (MIDI) protocol. The project is allocated on the TTSKY25b shuttle of the TinyTapeout project and is to be manufactured under the SkyWater 130 nm process. This report documents functionality as well as implementation and discusses design constraints, imposed by the limited physical space on the shuttle.

Acronyms

ASIC	application specific integrated circuit
DAC	digital to analog converter
DAW	digital audio workstation
FSM	finite statemachine
HDL	hardware description language
IC	Integrated Circuit
GM1	general MIDI level 1
GM2	general MIDI level 2
MIDI	musical instrument digital interface
MSB	most significant bit
PCB	printed circuit board
PDK	process development kit
UART	universal asynchronous reciever transmitter
USB	universal serial bus
SPN	scientific pitch notation
12ET	twelve tone equal temperament
I2S	inter-integrated sound

1. Introduction

MIDI is a versatile digital interface, which packs note expressions into a simple protocol. The note expression often originates from a MIDI-controller (source) and contains information about which key on the pianoroll is affected and how it is affected. A key thing to notice is, that the note expression itself does not contain any information about how the audiosignal sounds. The synthesis of the waveform is handled by a MIDI-instrument (sink) and translates the note expression into a audio timesignal. Figure 1 shows how a MIDI-keyboard might interface with a MIDI compatible synthesizer.

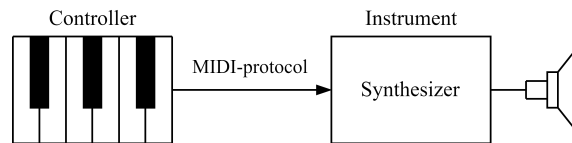


Figure 1: Setup of a controller-instrument-pair communicating via the MIDI-protocol.

The project implements such a MIDI-instrument and converts incoming MIDI-signals into audible squarewaves. The project is written in the hardware description language (HDL) Verilog specifically to be built as a digital application specific integrated circuit (ASIC) design by the open source LibreLane toolchain. The resulting chip will be taped out in collaboration with the Tiny-Tapeout project.

The Tiny-Tapeout project makes manufacturing ASIC-designs for individuals feasible by combining multiple smaller designs on a single chip under the usage of an open source process development kit (PDK). The chip space is sectioned in tiles, where each tile has around $160 \times 100 \mu\text{m}$ of available space. The project on hand occupies one tile. The active run at this time, TTSKY25b, manufactures the chip under SkyWater 130 nm PDK ruleset.

2. Digital Instruments and Music Theory

Before discussing the design, a few fundamentals of music theory have to be considered and how implementing a digital instrument in silicon is sensible. The MIDI Manufacturers Association defines the general MIDI level 1 (GM1) and its extension the general MIDI level 2 (GM2) standard to provide guidelines for developing MIDI compatible instruments.

2.1 The MIDI-Protocol

The MIDI-protocol is based on a serial asynchronous data transmission, where at most three bytes are sent in succession, with a dataformat as shown in Table 1. It is noted, that not all commands require a second data byte which is why it is specified in brackets as an optional byte. In this report all required commands provide a second data byte, therefore a length of three bytes will be assumed from now on.

Table 1: Dataformat of a MIDI-protocol transmission

Byte 1		Byte 2	Byte 3
Statusbyte			
Command	Channel	Databyte 1	[Databyte 2]

Table 2: Excerpt from the MIDI 1.0 specification message summary for channel voice messages [5].

$N = nnnn \in [0; 15]$ maps to a MIDI channel number 1-16.

$K = kkkkkk \in [0; 127]$ is the note number. (e.g. which key on a piano is pressed/released)

$V = vvvvvv \in [0; 127]$ is the velocity (how fast a note is pressed/released)

Status	Databytes	Description
1000nnnn	0kkkkkkk 0vvvvvvv	Note Off event. This message is sent when a note is released (ended).
1001nnnn	0kkkkkkk 0vvvvvvv	Note On event. This message is sent when a note is pressed (started).

The first byte combines two four-bit values representing a command and a channel number. Depending on the command code, the following databytes can be interpreted accordingly by a MIDI-sink listening on a pre-set channel. The most important and only command codes used in this project are listed in Table 2. A seven bit note number (denoted as $kkkkkkk$ in Table 2) maps to the piano keys of a twelve tone equal temperament (12ET), where the middle C (C_4) is equivalent to a MIDI note number of 60 in decimal [4, sec 2.7.1.1].

2.2 Instrument Tuning

A musical octave is an interval of two notes, where the frequency of the higher note is exactly twice as high as the lower note. The 12ET is a system which divides this octave into twelve notes, which are referred to in scientific pitch notation (SPN) as letters from A to G with accidentals (b, #) and an octave number n as shown in Table 3.

Table 3: SPN Nomenclature for notes in a 12 tone tempered octave, where n is the octave-number.

B_{n-1}	C_n	$\frac{C_{\sharp n}}{D_{\flat n}}$	D_n	$\frac{D_{\sharp n}}{E_{\flat n}}$	E_n	F_n	$\frac{F_{\sharp n}}{G_{\flat n}}$	G_n	$\frac{G_{\sharp n}}{A_{\flat n}}$	A_n	$\frac{A_{\sharp n}}{B_{\flat n}}$	B_n	C_{n+1}
-----------	-------	------------------------------------	-------	------------------------------------	-------	-------	------------------------------------	-------	------------------------------------	-------	------------------------------------	-------	-----------

These notes are equally tempered, meaning the frequency-ratio of two adjacent notes is constant for all musical notes. For an octave of 12 notes, this ratio equates to $\sqrt[12]{2} \approx 1.05946$, leading naturally to a logarithmic scale.

To assert actual frequency values to the notes, a pitch standard has to be decided on. GM2 suggests the ISO-16 pitch standard, where note number 69 in decimal (A_4) is tuned to 440 Hz [4, sec. 2.7.1.1]. With this reference frequency, the absolute frequency of all notes can be calculated by multiplying or dividing by the defined ratio, yielding Equation 1 for the note frequency $f(K)$ in dependence of the MIDI note number K .

$$f(K) = 440 \text{ Hz} \cdot (\sqrt[12]{2})^{(K-69)} = 440 \text{ Hz} \cdot 2^{(K-69)/12} \quad (1)$$

2.3 Instrument Characteristics

The frequency in Equation 1 yields the fundamental frequency of the sound oscillation generated by the instrument. However, the characteristic sound an instrument produces is caused by overtones relative to its fundamental. For example, the Grand Piano [1] in the digital audio workstation (DAW) Ableton Live (see Used Software) is characterised by overtones as seen in Figure 2.

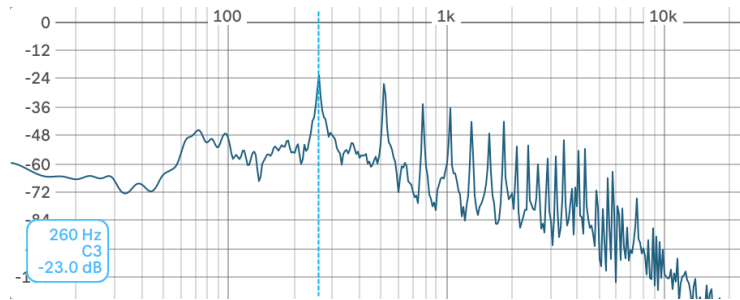


Figure 2: Overtones of a grand piano instrument playing a middle C (C3 in Lives notation).

Abscissa: Audiolevel in dB relative to the fullscale range of the audio engine.

Ortinate: Frequency in Hz.

A classical method to characterise the voice of a synthesizer is by using a wavetable. A wavetable is a discrete list of values representing the waveform amplitude over the span of one period (Figure 3). To synthesize the audio waveform, the table is being iterated at a certain speed, such that one loop spans over the period of the fundamental frequency.

To implement this in the design, the output capabilities of the ASIC have to be considered. As the design is purely digital, the output levels of the chip can only assume the logic levels high and low, which leaves following options:

1. Feed an audio stream to an external DAC using serial connectivity such as I2S.
2. Use all output pins as a parallel interface to drive a parallel DAC asynchronously.
3. Have the digital output be the synthesized waveform.

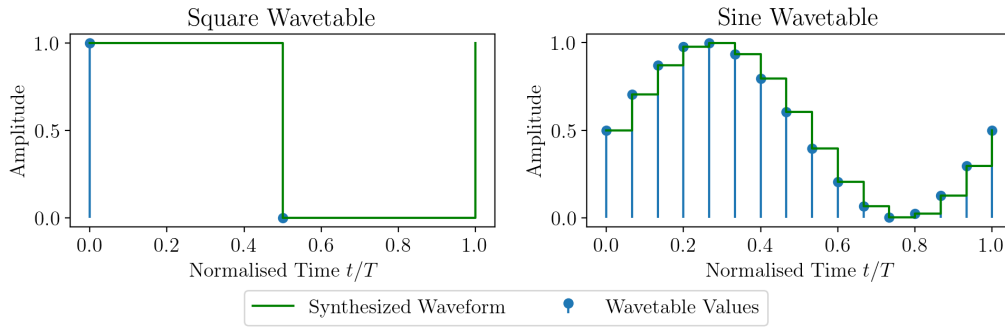


Figure 3: Wavetables for a 1-bit squarewave (left) and a 16-bit sinewave (right)

For this design the latter option is chosen, which has several benefits. Using a single digital output directly as the audio waveform is equivalent to a 1-bit wavetable, resulting in a squarewave. For a 1-bit wavetable the output can simply be toggled every half period.

The spectrum of an ideal squarewave with infinite runtime is defined by a fourier series in Equation 2 containing odd multiples

$$\omega_n = \frac{2\pi}{T}n, \quad n = 1, 3, 5, \dots$$

of the fundamental frequency ω_1 . The amplitude for harmonics of higher order declines hyperbolically [6], leading to a spectrum as shown in Figure 4.

$$s(t) := \sum_{n=1,3,5,\dots}^{\infty} \frac{4}{n\pi} \sin(\omega_n t) \quad \circ\text{---} \quad \|S(j\omega)\| := \sum_{n=1,3,5,\dots}^{\infty} \frac{4}{n} \delta(\omega - \omega_n) \quad (2)$$

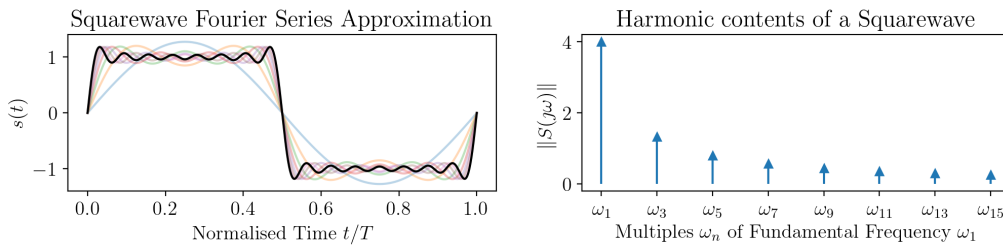


Figure 4: Approximation of a squarewave by adding odd harmonics one by one

2.4 Polyphony

It is common for instruments to play two notes simultaneously, which is caused by receiving multiple note-on commands without turning already active MIDI notes off. An important

parameter is the number of voices the instrument can produce. An instrument is considered polyphonic, if the number of voices is at least two. In this case, each voice is represented by a individual squarewave oscillators. One of the main challenges in this design is to reduce the area for the implementation of a voice oscillator to achieve a high polyphony count.

The audiosignal is directly sent to a digital pin. However, overlapping squarewaves of differing phase and frequency, again produce an analog signal (shown in Figure 5), which cannot be output by a single pin. Therefor, each voice has its own output pin. The individual voices are to be summed together by external circuitry, for example by an operational amplifier circuit (see section 8.3).

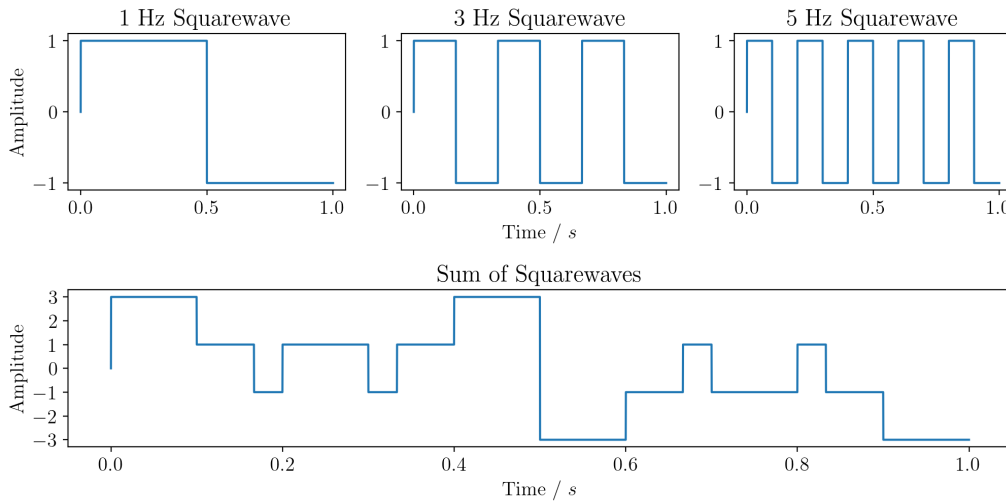


Figure 5: Individual Squarewaves summed up to an analog signal

2.5 MIDI Physical Layer

The MIDI protocol was originally transmitted asynchronously over a differential pair, which was daisychained between MIDI devices using 5 pole DIN connectors. With rising popularity of the universal serial bus (USB), MIDI devices adapted to this standard. Nowadays most MIDI devices are able to communicate via USB to a host device. Therefor MIDI instruments can also be controlled via USB under the usage of a USB to UART serial interface as it is intended with this design.

2.5.1 UART

The universal asynchronous reciever transmitter or UART is a simple interface which allows two devices to communicate over a serial connection. The devices are connected as shown in Figure 6. Each device can initiate a communication on its own behalf, without the acknowledgement of the other device, allowing for simultaneously sending

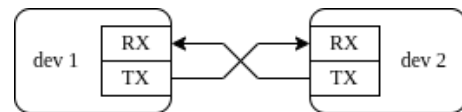


Figure 6: Wiring of two devices communicating via a UART connection

and receiving data. The connection does not feature a clock line, which indicates an asynchronous communication. The timing for the period of individual bits is determined by a baudrate, which has to be specified in both devices. For MIDI devices, the standard baudrate is 31250 [3].

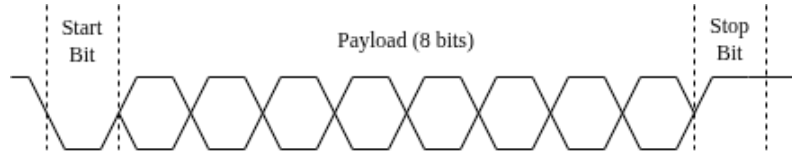


Figure 7: Dataframe of a UART communication

On idle, the dataline is normally set to a logic level *High* by the TX output. To initiate a data transfer, the trasmitter pulls the communication line low for one bit-period ($\frac{1}{\text{baudrate}}$). Then, eight bits are sent with the specified baudrate followed by a stop bit, which pulls the communication line high again. A complete datatransmission is shown in Figure 7.

It is also possible to introduce a parity bit for error correction and/or specify a payload with sizes other than 8 bit. This is not relevant in this project and will therefor not be explained in further detail.

Using a UART connection is very practical for transmitting MIDI signals between a controller and an instrument, as the serial MIDI-Data is compatible with the UART-payload size and does not require additional wires for e.g. clock or select signals.

3. System Overview

The source code can be found under the following link:

🔗 <https://github.com/s-grundner/LVA-EIC-KV>

3.1 Pinout and Top Level Module

The project features four voices implemented as individual squarewaves oscillators. The output of each oscillator is routed to the outputpins `uo[0]...uo[3]`. Output `uo[7]` yields a PWM Signal, which encodes the number of actively running oscillators. This output may be used to control the gain of external circuitry when mixing the oscillator outputs together. The project has one input `ui[3]` to which the MIDI signal has to be supplied to. This pin is also connected to the UART-TX output of the RP2040 Microcontroller, which is populated on the TinyTapeout test PCB. The RP2040 may be used to emulate a MIDI controller or to interface the instrument via USB. Figure 8 summarises the in- and outputs of the ASIC.

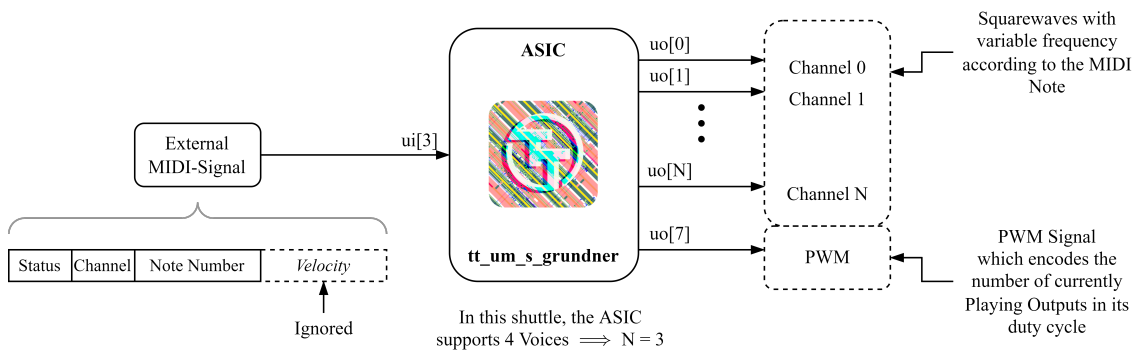


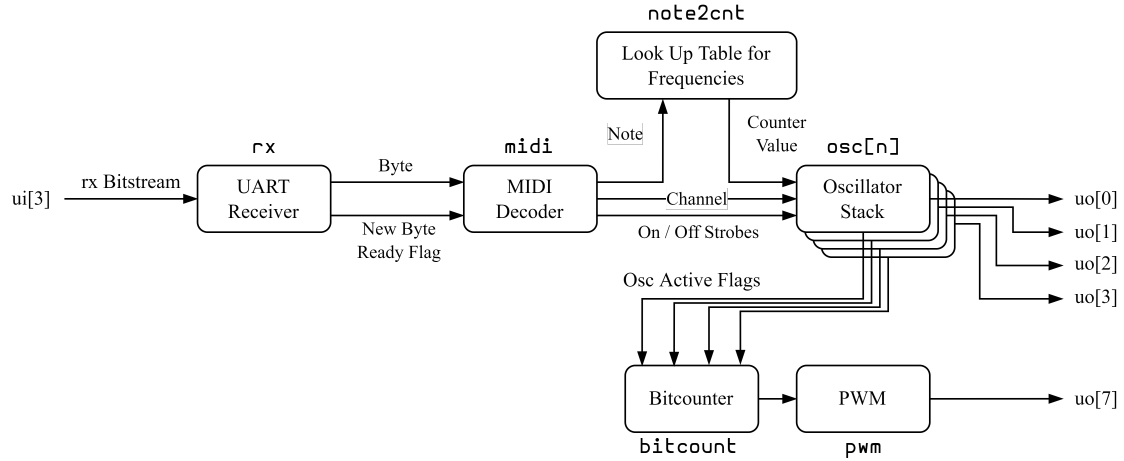
Figure 8: Input and output visualisation and description

3.2 Block Diagram

An abstract overview of the project implementation can be seen in Figure 9. The block-titles outside of each block represent the Verilog module name coherent with the code-repository.

The signal flow starts at the input, where the MIDI messages are received via UART. The bits of the input stream are collected into bytes, which are then fed into a MIDI decoder. The MIDI decoder extracts the note- and channel number and if a on or off command is received.

The note number indexes a lookup table which returns a counterbase for the corresponding frequency. Each oscillator in the stack is listening on its own MIDI-channel. The note-on/note-off commands only affect the oscillator listening on this channel. Upon receiving a note on command, the counterbase value at the input of an oscillator is latched and a

Figure 9: Abstract Blockdiagram of the Synthesizer (`synth` module)

counter starts counting repeatedly in this time-base. The output of the oscillator is then toggled at each counter overflow, which generates the squarewave output.

3.3 Layout

To demonstrate the density of the layout, an image of the generated GDS is shown in Figure 10.

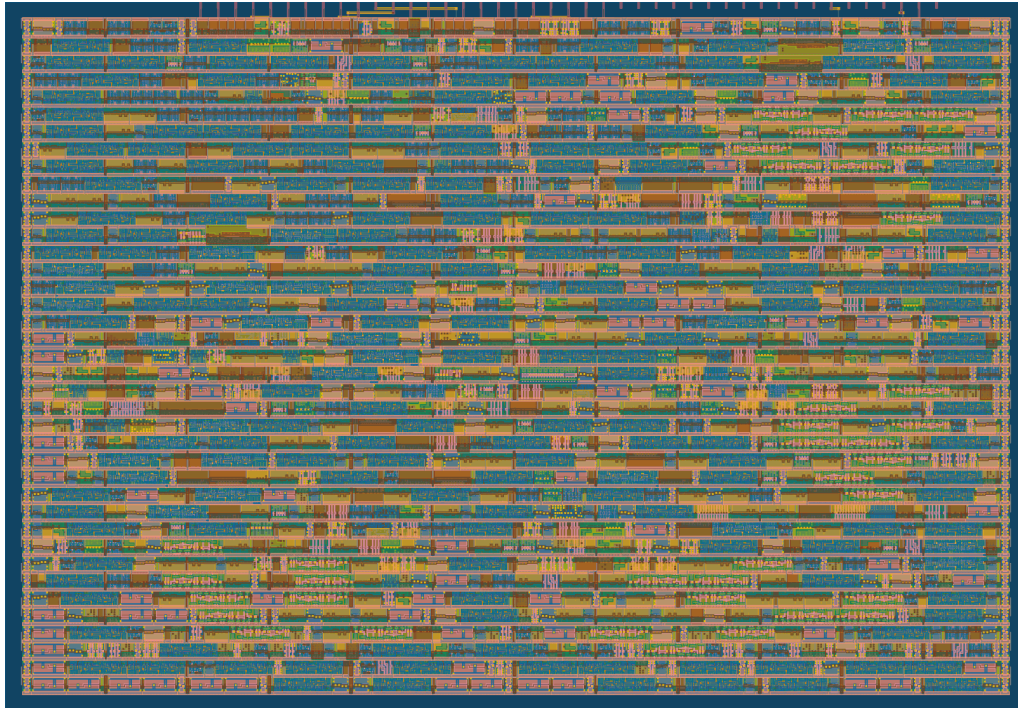


Figure 10: Layout of the Tile

The routing statistics generated by the TinyTapeout pipeline are listed in Table 4. Especially a space utilisation of 74.275 % shows, that space-optimization was crucial. Per

default, the maximum density is set to 60%, which had to be adjusted to fit more features. The documentation of the TinyTapeout project stated, that a density of up to 80% worked in previous shuttles without issues. The layout as it is shown in Figure 10 is comprised of logic cells which are predefined by the PDK. Which logic cells are generated from the Verilog code can be seen in Table 5.

Table 4: Routing statistics

Utilisation / %	Wire length / μm
74.275	23342

Table 5: Used Logic cells of the PDK and the number of occurrences

Category	Cells	Count
Fill	decap fill	733
Combo Logic	a22o a41o and2b a2bb2o o22ai a31o a22oi o21ai a221o o22a o2111ai a211o or4b a21oi a21o o211a and3b a311o o21a o2bb2a o221a a2111o and4b o211ai or4bb a211oi or3b a31oi o41ai o21ba a21bo a32o a221oi o2111a and4bb o32a	228
Tap	tapvpwrvgnd	225
Flip Flops	dfrtp dfstp	191
Multiplexer	mux4 mux2	180
Misc	conb dlymetal6s2s dlygate4sd3	98
AND	and3 and2 and4 a21boi	95
Buffer	clkbuf buf	79
NOR	nor2 xnor2	59
Inverter	inv	49
OR	or2 xor2 or3 or4	48
NAND	nand2 nand3 nand2b	32
Clock	clkinv	10
1069 total cells (excluding fill and tap cells)		

3.4 Global Settings

Certain parameters are reused throughout the entire project. These parameters are collected into a `global.v` file, which is included in every verilog source file. The defined global variables are listed in Table 6. How these parameters are chosen will be documented throughout the report.

Table 6: Content of `global.v` in a tabular format

Variable	Value	Description
<code>F_CLK_HZ</code>	<code>3_496_503</code>	Global Clock Speed which has to be supplied to the IC
<code>F_CLK_PERIOD_NS</code>	<code>1_000_000_000 / `F_CLK_HZ</code>	
<code>F_BAUD</code>	<code>31250</code>	
<code>F_BAUD_PERIOD_NS</code>	<code>1_000_000_000 / `F_BAUD</code>	
<code>MIDI_PAYLOAD_BITS</code>	<code>8</code>	
<code>MIDI_NOTE_BW</code>	<code>7</code>	
<code>OSC_VOICES</code>	<code>4</code>	MAX 7
<code>OSC_VOICES_BW</code>	<code>\$clog2(`OSC_VOICES+1)</code>	
<code>OSC_CNT_BW</code>	<code>16</code>	Counter bit width
<code>OSC_ROM_BW</code>	<code>7</code>	Note ROM bit width

3.5 Modules

A quick overview and brief description on the Verilog modules is given in Table 7. How the modules are connected can be seen in Figure 9.

Table 7: Overview of Verilog modules

Module	Description
<code>tt_um_s_grundner</code>	Tiny Tapeout top level module
<code>synth</code>	Combines reciever, lookup table und oscillator stack
<code>osc</code>	generates a squarewave of specified frequency upon receiving a note on command. Disables output for note off commands
<code>midi</code>	Extracts note on/off commands, channel and note number from incoming bytes.
<code>rx</code>	RX frontend of midi input. 31250 baud, 8 data bits, no parity
<code>note2cnt</code>	Convert note numbers to counter base via LUT
<code>counter</code>	Counts up on each clock cycle
<code>bitcount</code>	Returns the number of high bits in the input vector
<code>pwm</code>	PWM output for encoding the number of active Oscillators. The Frequency of the PWM output is: $f = f_{CLK}/N_{VOICES}$

4. Receiver

As UART is a common interface, there are already several implementations available in Verilog. For this project, an open source implementation¹ under the MIT license was used as a starting point and has then been adjusted to the project. As the project only receives data and does not send any, only the UART-receiver has to be implemented.

Listing 1: Receiver module ports

```

1 module rx (
2     input wire clk_i,
3     input wire nrst_i,
4     input wire rxData_i,
5     output reg dataReady_o,
6     output reg [`MIDI_PAYLOAD_BITS-1:0] midiData_o
7 );

```

4.1 Statemachine

At the core of the receiver is a finite state machine (FSM) (Figure 11) with the four states a UART transmission can assume: IDLE, START, RECV and STOP. A combinatorial process decides, which state is registered with the next clock cycle as shown in Listing 2.

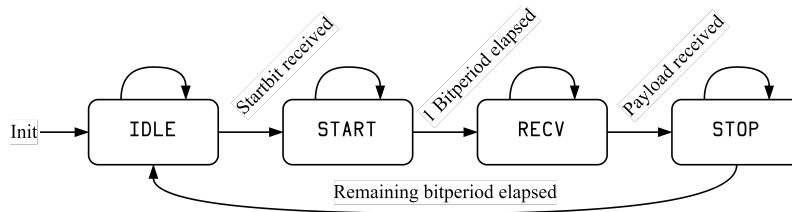


Figure 11: State machine diagram for the receiver

Listing 2: Combinatorics to select next FSM state

```

1 // Next State Conditions
2 wire nextBitReady = (cycleCounter == COUNT_REG_LEN'(CYCLES_PER_BIT)) ||
3                     (fsmState == FSM_STOP) &&
4                     (cycleCounter == COUNT_REG_LEN'(CYCLES_PER_BIT_HALF));
5 wire payloadDone = (bitCounter == `MIDI_PAYLOAD_BITS);
6
7 // Select Next State
8 always @(*) begin : nextFsmState_p
9     case (fsmState)
10         FSM_IDLE: nextFsmState = rxDataReg0 ? FSM_IDLE : FSM_START;
11         FSM_START: nextFsmState = nextBitReady ? FSM_RECV : FSM_START;
12         FSM_RECV: nextFsmState = payloadDone ? FSM_STOP : FSM_RECV;
13         FSM_STOP: nextFsmState = nextBitReady ? FSM_IDLE : FSM_STOP;
14         default: nextFsmState = FSM_IDLE;
15     endcase
16 end

```

¹<https://github.com/ben-marshall/uart>

1. **IDLE**: With each clock cycle, the input is latched in `rxDataReg` (Listing 3). The data register is normally high in the idle state. When a startbit is detected, so the `rxDataReg` wire assumes a low state, the next state is set to **START**.
2. **START**: The counter `cycleCounter` divides the frequency into the baudrate defined in `global.v` (Listing 4). After a bit-period has elapsed, the next state is set to **RECV**, as the period of the start bit has now elapsed.
3. **RECV**: A second counter `bitCounter` counts the number of sampled databits at the input (Listing 5). When the expected payload is recieved, the next state is set to **STOP**.
4. **STOP**: The cycle counter counts the remaining half bit period and sets the next state to **IDLE**.

Listing 3: Register the input wire with each clock cycle. To break up long combinatorial chains before registering, the received data is internally latched a second time.

```

1  always @(posedge clk_i or negedge nrst_i) begin : rxBuffer_p
2      if (!nrst_i) begin
3          rxDataReg0 <= 1'b1;
4          rxDataReg1 <= 1'b1;
5      end else begin
6          rxDataReg0 <= rxDataReg1;
7          rxDataReg1 <= rxData_i;
8      end
9  end

```

Listing 4: Increment the cycle counter. The counter is incremented in each clockcycle when not in the **IDLE** state. The counter is reset after the counter reaches the maximum cycles per bit count. After the counter overflows, a bit period defined by the baudrate is elapsed.

```

1  always @(posedge clk_i or negedge nrst_i) begin : countCycles_p
2      if (!nrst_i) begin
3          cycleCounter <= COUNT_REG_LEN'(0);
4      end else if (nextBitReady) begin
5          cycleCounter <= COUNT_REG_LEN'(0);
6      end else if (fsmState == FSM_START || fsmState == FSM_RECV || fsmState ==
7      FSM_STOP) begin
8          cycleCounter <= cycleCounter + COUNT_REG_LEN'(1);
9      end
10 end

```

Listing 5: Increment the bit counter. The counter increments when the cycle counter reaches its maximum (a new bit is ready to be sampled) and the FSM is in the **RECV** state. The counter is reset when the FSM leaves the **RECV** state.

```

1  always @(posedge clk_i or negedge nrst_i) begin : countBit_p
2      if (!nrst_i) begin
3          bitCounter <= 4'b0;
4      end else if (fsmState != FSM_RECV) begin
5          bitCounter <= 4'b0;
6      end else if (fsmState == FSM_RECV && nextBitReady) begin
7          bitCounter <= bitCounter + 4'b1;
8      end
9  end

```

To acquire the data, the input is latched into a sample register in the middle of a bit period (Listing 6). When the bit period has elapsed, the sampled bit is then collected in the `midiData` byte (Listing 7), which is then assigned to the output of the module. Whenever the whole payload is received, the `dataReady_o` strobe² is pulsed (Listing 8), which is required by the MIDI decoder.

Listing 6: Sample a bit in the middle of a bit period.

```

1  always @(posedge clk_i or negedge nrst_i) begin : sampleBit_p
2      if (!nrst_i) begin
3          sampledBit <= 1'b0;
4      end else if (cycleCounter == COUNT_REG_LEN'(CYCLES_PER_BIT_HALF)) begin
5          // Take sample in the middle of a bit
6          sampledBit <= rxDataReg0;
7      end
8  end

```

Listing 7: Collect sampled data into a MIDI byte. The latest bit is always inserted at the MSB position. Already received bits are shifted right.

```

1  integer i;
2  always @(posedge clk_i or negedge nrst_i) begin : updRxBuffer_p
3      if (!nrst_i) begin
4          midiData <= `MIDI_PAYLOAD_BITS'b0;
5      end else if (fsmState == FSM_IDLE) begin
6          midiData <= `MIDI_PAYLOAD_BITS'b0;
7      end else if (fsmState == FSM_RECV && nextBitReady) begin
8          midiData[`MIDI_PAYLOAD_BITS-1] <= sampledBit;
9          for (i = `MIDI_PAYLOAD_BITS - 2; i >= 0; i = i - 1) begin
10             midiData[i] <= midiData[i+1];
11         end
12     end
13 end

```

Listing 8: Output a data ready strobe whenever a transmission is complete.

```

1  always @(posedge clk_i or negedge nrst_i) begin : dataReady_p
2      if (!nrst_i) begin
3          dataReady_o <= 1'b0;
4      end else if (fsmState == FSM_STOP) begin
5          dataReady_o <= payloadDone;
6      end else begin
7          dataReady_o <= 1'b0;
8      end
9  end

```

²A strobe is a signal, which is high until the next rising edge of the clock signal.

5. MIDI-Decoder

The MIDI decoder is responsible for extracting MIDI command and channel as well as the note number from the received bytes. The in- and outputs of this module can be seen in Listing 9.

Listing 9: MIDI module ports.

```

1 module midi (
2     input wire clk_i,
3     input wire nrst_i,
4     input wire midiByteValid_i,
5     input wire [`MIDI_PAYLOAD_BITS-1:0] midiByte_i,
6     output reg [`OSC_VOICES_BW-1:0] ch_o,
7     output reg [`MIDI_NOTE_BW-1:0] note_o,
8     output reg noteOnStrb_o,
9     output reg noteOffStrb_o
10 );

```

5.1 Statemachine

Here again, a finite statemachine shown in Figure 12 underlies the MIDI decoder. The decoder has four states, representing which type of byte is currently received. IDLE, CMD and NOTE (Listing 10). The FSM state switches states upon receiving a data ready strobe (`midiByteValid_i`) from the UART receiver. The input byte is also latched on this event (Listing 11) and is then being processed according to the state.

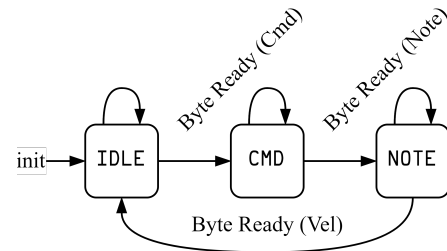


Figure 12: MIDI decoder statemachine

Listing 10: States of the finite statemachine. The next state will be immediately set, the assignment condition of the next state to the current state is handled in a register process.

```

1 always @(*) begin : nextFSM_p
2     case (fsmState)
3         FSM_IDLE: nextFsmState = FSM_CMD;
4         FSM_CMD: nextFsmState = FSM_NOTE;
5         FSM_NOTE: nextFsmState = FSM_IDLE;
6         default: nextFsmState = FSM_IDLE;
7     endcase
8 end

```

The velocity data itself is ignored, however, it still needs to be present at the input, because the data ready strobe of the last byte is needed to switch into the IDLE state. For most instruments, the velocity is interpreted as the loudness of the tone. As the digital output only has one amplitude level, this value is obsolete.

Listing 11: Register the next FSM state aswell as the input byte, when the input byte is valid.

```

1  always @(posedge clk_i or negedge nrst_i) begin : midiByteReg_p
2      if (!nrst_i) begin
3          midiByteReg <= {`MIDI_PAYLOAD_BITS{1'b0}};
4          fsmState <= FSM_IDLE;
5      end else if (midiByteValid_i) begin
6          midiByteReg <= midiByte_i;
7          fsmState <= nextFsmState;
8      end
9  end

```

1. **IDLE**: The FSM resides in the idle state until the first byte is ready. On this event, the state is switched to CMD.
2. **CMD**: The received byte is split into the channel number and the command in Listing 12 according to Table 1. When the next byte is available, the state switches to NOTE.
3. **NOTE**: The previously received command is now available for evaluation. The output strobes `noteOnStrb_o` and `noteOffStrb_o` are assigned according to the command in Listing 13 and will notify the oscillatorstack, if the current channel output needs to be turned on or off. The received byte is assigned to the note output in Listing 14.

Listing 12: Deconstruct the status byte into channel number (upper 4 bits) and command (lower 4 bits). Forwards the channel to the output and latches the command internally to be used by the next state.

```

1  always @(posedge clk_i or negedge nrst_i) begin : statusEval_p
2      if (!nrst_i) begin
3          cmd <= {CMD_BW{1'b0}};
4          ch_o <= {`OSC_VOICES_BW{1'b0}};
5      end else if (fsmState == FSM_CMD) begin
6          cmd <= midiByteReg[7:4];
7          ch_o <= midiByteReg[`OSC_VOICES_BW-1:0];
8      end
9  end

```

Listing 13: Forwards the MIDI byte ready strobe to either the note-on or note-off strobe, according to the received command.

```

1  always @(posedge clk_i or negedge nrst_i) begin : noteOnOffStrb_p
2      if (!nrst_i) begin
3          noteOnStrb_o <= 1'b0;
4          noteOffStrb_o <= 1'b0;
5      end else if (fsmState == FSM_NOTE && midiByteValid_i) begin
6          noteOnStrb_o <= (cmd == CMD_NOTE_ON) ? 1'b1 : 1'b0;
7          noteOffStrb_o <= (cmd == CMD_NOTE_OFF) ? 1'b1 : 1'b0;
8      end else begin
9          noteOnStrb_o <= 1'b0;
10         noteOffStrb_o <= 1'b0;
11     end
12 end

```

Listing 14: The received byte is forwarded to the note output of the module.

```

1  always @(posedge clk_i or negedge nrst_i) begin : note_p
2      if (!nrst_i) begin
3          note_o <= {'OSC_ROM_BW{1'b0}};
4      end else if (fsmState == FSM_NOTE) begin
5          note_o <= midiByteReg[`MIDI_NOTE_BW-1:0];
6      end
7  end

```

5.2 Limitations

According to the GM1 specifications, a sound generating instrument must implement a set of features to ensure compatibility with various controller outputs / MIDI-sequences [2]. This project does not fully comply with GM1 as even the smallest feature set requires a much more complex MIDI statemachine, which is beyond the goals of this project. Because of this, directly connecting a MIDI-controller to the IC might lead to an invalid state of the MIDI-state machine, as not all MIDI-Messages are guaranteed to be 3 bytes long. Edge cases which are required to be handled by the device are evaluated in section 7

A particular feature of the MIDI decoder, which is normally not found in typical applications, is that polyphony is only achieved by sending parallel inputs onto different MIDI channels. To clarify this, an example for two input cases is given:

- A:** A key is held down, sending a note-on command on channel 1. The key is synthesized by the IC, outputting a squarewave on pin `uo[0]`. Upon pressing another key, which is also sent on channel 1, the IC overrides the output 1, playing the most recent note on the channel. This is even the case when the remaining oscillators are in the idle state.
- B:** A key is held down, [...]. Upon pressing a key which is sent onto a different channel than 1, e.g. channel 2, the output `uo[0]` continues to play the old note and the newly pressed note is being sent onto `uo[1]`. This is the way in which polyphony is achieved.

This behaviour is called Voice-Stealing. Voice-Stealing is implemented by instruments, so that if the number of keys pressed by the controller exceeds the polyphonic count of the instrument, the instrument switches the earliest played note off and allows the newly pressed key to play a note. This static channel routing approach was necessary to reduce the logic of the MIDI-decoder.

6. Synthesizer

How is the Frequency Generated?

Count up a defined Number of steps

For each Count, a time interval of $1/f_{clk}$ elapses.

The Lowest Frequency requires the highest number of counts

The Task is to (space) efficiently determine the Counter Period from the MIDI-Note number

6.1 Frequency Calculation and Generation

Limitations:

- Frequency Resolution determined by Clock Frequency - Divisions and Modulo operations are Expensive.

Choosing a Clock Frequency

- See reasoning in Py Notebook

Externalize Calculation Logic from the Oscillator Stack

Compare optimisations

- Performing an actual Twelvetone Temperament Calculation on Hardware
- Lookup Table: Save the Countervalues of the Twelve Notes on the Octave to achieve Most precise Counterperiod. What Calculations are required by the oscillators
- Lookup Table: Reduce Bitwidth and therefor required LUT Size. Compare Size Reduction

Impact of Frequency Deviation

- Systematic error of frequencies from reducing counter resolution. Deviations and calculations from Py-Notebook.
- Spectrum and Harmonics

Are the Deviations Audible?

- Heavily Dependent on the individual Person (Hearing Curve)
- Document Experiment in Ableton Live

6.2 Oscillatorstack

Routing a MIDI a Channel to a fixed Oscillator.

Modularity by a Generating Function: Option to increase number of voices for larger space or future Design optimisations.

7. System Testing

Edge Cases

Limitations. Which tests are omitted. Correctness of the midi Input Byte Order for example.

CocoTB Testresults, Design Simulation+validation, Measurements, Waveforms

8. External Hardware

8.1 MIDI-Source

As the device does not fully comply to the GM-1 specs, it is best to provide MIDI inputs from a controlled environment. Pluggin in a full-fledged midi controller directly might lead to unwanted behaviour.

Example of a USB-MIDI controller.

In Principle Direct connection is Possible but a Powersupply is required.

Setting up a Virtual MIDI-Source with a DAW, Hairless MIDI and

PMOD as a PHY for the native Differential midi connection.

8.2 Input Side

Digital Frontend converting simultaneous Notes into individual Channel

8.3 Output Side

Audio Mixing Circuit, Gain Control, Speaker Driver

Option for mild subtractive Sound-Design

- Squarewaves provide all odd harmonics of the fundamental frequencies, which can be filtered by an external system to create different Sounddesigns

- optimal Would be a Sawtooth as it contains all harmonics but violated the digitality of the ASIC Output.

9. Used Software

appendix: Setting up a development env.

Table 8: List of used software for development and comissioning this project

Name	Version	Description	Usage
Ableton Live 11 Suite	11.3.43	DAW	

References

[1] Ableton AG. Live Grand Piano. <https://www.ableton.com/en/packs/grand-piano>. Accessed: 2026-01-02.

[2] The MIDI Manufacturers Association. General MIDI System Level 1 Specification. <https://midi.org/general-midi-level-1>, 1996. Accessed: 2025-12-25.

[3] The MIDI Manufacturers Association. MIDI 1.0 Detailed Specification. <https://midi.org/midi-1-0-detailed-specification>, 1996. Accessed: 2026-01-13.

[4] The MIDI Manufacturers Association. General MIDI System Level 2 Specification. <https://midi.org/general-midi-level-2-2>, 2007. Accessed: 2025-12-25.

[5] The MIDI Manufacturers Association. Summary of MIDI 1.0 Messages. <https://midi.org/summary-of-midi-1-0-messages>, 2020. Accessed: 2025-12-25.

[6] Weisstein, Eric W. Fourier Series–Square Wave. From MathWorld–A Wolfram Resource. <https://mathworld.wolfram.com/FourierSeriesSquareWave.html>. Accessed: 2026-01-02.

Wikipedia References

- W MIDI - Wikipedia
- W MIDI tuning standard - Wikipedia
- W Scientific pitch notation - Wikipedia
- W 12 equal temperament - Wikipedia
- W A440 (pitch standard) - Wikipedia